

PROMPT ENGINEERING

Detailed Training Notes — Prompt Structure, Zero/One/Few-Shot Learning, System & Role Prompts, Step-Back Prompting and Reasoning Strategies

Purpose: These notes reorganize the training session into a clean study sequence. They explain what each prompting technique does, when to use it, its limitations, and practical examples.

Learning Roadmap

- What prompt engineering is and why it matters
- Zero-shot, one-shot, and few-shot prompting
- System, contextual, and role prompting
- How constraints reduce irrelevant outputs
- Step-back prompting
- Chain-of-thought and safer reasoning-oriented prompting
- Generation controls: max tokens, temperature, top-p, and top-k
- Structured output such as JSON
- Prompt templates, practical examples, common mistakes, and interview answers

1. What Is Prompt Engineering?

Prompt engineering is the practice of designing instructions, context, examples, constraints, and output requirements so that a language model is more likely to produce a useful response.

A prompt can contain several parts:

- **Task:** What should the model do?
- **Context:** What background information is relevant?
- **Role:** What perspective or behavior should the model adopt?
- **Examples:** What does a correct input-output pattern look like?
- **Constraints:** What must or must not appear?
- **Output format:** Paragraph, bullets, table, JSON, code, etc.

Key idea: A good prompt reduces ambiguity. It does not magically make a model factual or eliminate hallucinations.

Simple Prompt Framework

ROLE:

You are a technical recruiter.

TASK:

Review the resume for a Python developer role.

CONTEXT:

The role requires Python, SQL, AWS, and REST APIs.

CONSTRAINTS:

- Do not invent experience.
- Identify only evidence present in the resume.

OUTPUT:

Return:

1. Match score
2. Strong matches
3. Missing skills
4. Three improvement suggestions

2. Zero-Shot Prompting

Zero-shot prompting means asking the model to perform a task **without giving an example** of the expected input-output pattern.

Example:

Classify the following review as POSITIVE, NEUTRAL, or NEGATIVE.
Return only the label.

Review:

"The movie was too long and I would not watch it again."

Expected output: **NEGATIVE**

Use zero-shot when: the task is common, simple, and the instructions are already clear.

3. One-Shot Prompting

One-shot prompting gives the model **one example** before asking it to perform the actual task. The example demonstrates the desired pattern, structure, or style.

Convert a pizza order into valid JSON.

Example:

Input: I want a small pizza with cheese and pepperoni.

Output:

```
{
  "size": "small",
  "toppings": ["cheese", "pepperoni"]
}
```

Now convert:

I want a large pizza with mushrooms and olives.

The example helps the model infer the expected schema and formatting.

4. Few-Shot Prompting

Few-shot prompting provides **multiple examples**. It is useful when the task has a pattern that is difficult to describe only with instructions.

Convert each order into JSON.

Example 1:

Input: Small pizza with cheese.

Output: {"size": "small", "toppings": ["cheese"]}

Example 2:

Input: Large pizza with mushrooms and olives.

Output: {"size": "large", "toppings": ["mushrooms", "olives"]}

Now:

Input: Medium pizza with pepperoni and onions.

Output:

Why examples help: They demonstrate the expected mapping from input to output. However, poor examples can also teach the model the wrong pattern.

5. Zero-Shot vs One-Shot vs Few-Shot

Technique	Examples Given	Best Use
Zero-shot	None	Simple or familiar tasks with clear instructions
One-shot	One	Demonstrating a specific format or style
Few-shot	Several	Teaching a pattern through examples

6. System Prompting

A system-level instruction defines the model's broad behavior, purpose, rules, and constraints for an interaction or application.

You are a sentiment classification assistant.

Rules:

- Classify each review as POSITIVE, NEUTRAL, or NEGATIVE.
- Return only the uppercase label.
- Do not provide an explanation.

User input: *"The story was predictable and the acting was disappointing."*

Output: **NEGATIVE**

Technical note: In an API or chat architecture, a true system message may have a special priority defined by the platform. Simply writing "System:" inside ordinary user text is not always equivalent to an actual system-level message.

7. Contextual Prompting

Contextual prompting gives the model relevant background information needed for the current task.

Context:

The customer is a beginner investor with a \$500 budget.
They prefer lower-risk, long-term investing.

Task:

Explain the difference between an individual stock and a broad-market ETF.

Constraints:

Use simple language. Do not provide personalized financial guarantees.

The context narrows the response toward the user's situation. Good context should be relevant; dumping unnecessary information can make the prompt longer and less focused.

8. Role Prompting

Role prompting asks the model to respond from a particular professional perspective or communication style.

Act as an experienced travel guide.

I am visiting Amsterdam and I mainly enjoy museums.
Recommend three places and briefly explain why each fits my interest.

Another example:

Act as a technical recruiter hiring an AI Engineer.

Review the following resume against the job description.
Do not invent skills or experience.

Identify:

- Strong matches
- Gaps
- Interview risks
- Improvements

What role prompting actually does: It guides perspective, terminology, priorities, and response style. It can narrow the task, but it does **not** guarantee expertise, truthfulness, or freedom from hallucination.

9. Constraints and Scope Control

A strong prompt often explicitly defines boundaries. This is usually more reliable than relying on a role alone.

- Specify what sources or context may be used.
- Tell the model not to invent missing information.
- Define the expected output format.
- Set length or detail requirements.
- Specify what to do when information is missing.

Use only the information in the supplied resume.

If a required skill is not mentioned, write "Not found in resume."

Do not infer years of experience.

Return the answer as a table.

10. Structured Output and JSON Prompting

Structured output is useful when another program must process the model response. JSON is a common format.

Extract the pizza order.

Return valid JSON using exactly this schema:

```
{
  "size": "small | medium | large",
  "toppings": ["string"]
}
```

Order:

"I want a large pizza with mozzarella and mushrooms."

Expected structure:

```
{
  "size": "large",
  "toppings": ["mozzarella", "mushrooms"]
}
```

Best practice: Define the schema explicitly. If the application requires strict machine-readable output, use platform-supported structured-output or schema-validation features when available rather than trusting prompt wording alone.

11. Step-Back Prompting

Step-back prompting asks the model to first consider a broader principle, background concept, or useful intermediate context before solving the specific task.

Instead of immediately asking:

Why do airplanes fly?

A step-back approach might be:

First explain the main physical forces that act on an airplane in flight.
Then use those principles to explain why airplanes can fly.

The broader question activates relevant concepts—such as lift, weight, thrust, and drag—before the specific explanation.

Training-session style example

The class example first asks for common themes or settings used in engaging first-person shooter games. The resulting themes are then supplied as context for a second prompt that asks the model to create a new storyline.

Pattern: Broader supporting question → useful context → specific target task

12. Chain-of-Thought (CoT) Prompting

Chain-of-thought prompting is associated with encouraging intermediate reasoning for multi-step problems. The training example uses arithmetic to illustrate solving a problem through intermediate steps.

Problem:

A store sells 12 boxes with 25 items in each box.
Calculate the total.

Useful response style:

- Identify the quantities.

- Apply the correct operation.
- Verify the result.
- Give the final answer.

Practical recommendation: In many applications, ask for a **concise explanation, calculation steps, assumptions, or verification** rather than requiring hidden internal reasoning. The goal is to obtain a useful, checkable answer—not an unnecessarily long reasoning transcript.

13. Step-Back vs Reasoning-Oriented Prompting

Technique	Main Idea	Example
Step-back	First identify broader principles or context	Explain forces on aircraft, then explain flight
Reasoning-oriented	Break a complex task into explicit, checkable stages	Identify values → calculate → verify → answer

14. Generation Controls Mentioned in the Training

The transcript also mentions token limits, top-p, and top-k. These are **generation parameters**, not prompting techniques themselves.

Maximum Output Tokens

Limits how many tokens the model may generate. A very small limit can cut off an answer.

Temperature

Controls randomness in token selection. Lower values generally produce more focused or repeatable outputs; higher values generally increase variation.

Top-p (Nucleus Sampling)

Restricts sampling to a dynamic set of likely tokens whose cumulative probability reaches the chosen threshold.

Top-k

Restricts sampling to the k highest-probability candidate tokens before selecting the next token.

Correction: Saying “top-k = 40 means the model gives the top 40 tokens” is misleading. It means the sampling process can restrict consideration to up to the 40 highest-probability candidates at that generation step.

15. A Strong General-Purpose Prompt Template

ROLE

You are a [relevant role].

GOAL

Your task is to [specific objective].

CONTEXT

[Only relevant background information.]

INPUT

[Data, question, document, or user request.]

REQUIREMENTS

- [Requirement 1]
- [Requirement 2]
- [Requirement 3]

CONSTRAINTS

- Do not invent missing facts.
- State uncertainty when necessary.
- Stay within the supplied scope.

OUTPUT FORMAT

Return:

1. ...
2. ...
3. ...

QUALITY CHECK

Before answering, verify that the response follows the requested format and does not contradict the supplied information.

16. Python Example — Building Prompt Messages

The following example demonstrates prompt structure without depending on a specific AI provider.

```
# Define the overall behavior of the assistant
system_message = '''
You are a movie sentiment classifier.

Rules:
- Allowed labels: POSITIVE, NEUTRAL, NEGATIVE
- Return only one label
- Do not add explanations
'''

# The actual task input from the user
user_message = '''
Classify this review:
"The movie was visually impressive, but the story was average."
'''

# Store the messages in a structured form
messages = [
    {
        "role": "system",
        "content": system_message
    },
    {
        "role": "user",
        "content": user_message
    }
]

# 'messages' can then be passed to the chat API or model interface
# supported by the AI platform you are using.
print(messages)
```

Code explanation:

- **system_message** defines the task rules and output constraints.
- **user_message** contains the actual item to classify.
- **messages** separates instructions from user input in a structured format.
- The exact API call depends on the AI platform and SDK being used.

17. Python Example — Few-Shot Prompt Construction

```
# A few-shot prompt teaches the expected input-output pattern
prompt = '''
Convert pizza orders to JSON.

Example 1
Input: Small pizza with cheese
Output: {"size": "small", "toppings": ["cheese"]}

Example 2
Input: Large pizza with mushrooms and olives
Output: {"size": "large", "toppings": ["mushrooms", "olives"]}

Now convert this order:
Input: Medium pizza with pepperoni and onions
Output:
'''

# Print the complete prompt before sending it to a model
print(prompt)
```

The examples act as demonstrations. The model is expected to continue the demonstrated pattern for the new input.

18. Common Prompting Mistakes

- **Vague task:** “Tell me about this” gives the model too much freedom.
- **Conflicting instructions:** Asking for both “one sentence” and “a detailed 1,000-word explanation.”
- **Too much irrelevant context:** More context is not automatically better.
- **Poor few-shot examples:** Incorrect examples can guide the model toward incorrect outputs.
- **Assuming a role guarantees expertise:** “Act as a doctor” does not make a model a licensed medical professional.
- **No output contract:** If format matters, specify it explicitly.
- **No uncertainty rule:** Tell the model what to do when information is missing.
- **Trusting prompts as security boundaries:** Prompt instructions alone are not a complete security or factuality solution.

19. Choosing the Right Technique

Situation	Useful Approach
Simple classification	Zero-shot + strict labels
Specific output format	One-shot or schema-based structured output
Complex pattern difficult to describe	Few-shot examples
Need a particular professional perspective	Role + explicit constraints
Task depends on background information	Contextual prompting

Situation	Useful Approach
Specific problem benefits from broader principles	Step-back prompting
Multi-step problem	Explicit decomposition, calculation, verification, and concise explanation

20. Interview-Ready Questions and Answers

Q: What is zero-shot prompting?

Zero-shot prompting asks a model to perform a task without providing examples. The model relies on the instruction and its learned capabilities.

Q: What is the difference between one-shot and few-shot prompting?

One-shot provides one demonstration of the desired pattern, while few-shot provides multiple demonstrations.

Q: Why use role prompting?

Role prompting guides the model toward a particular perspective, terminology, or response style. It should usually be combined with clear task instructions and constraints.

Q: What is contextual prompting?

Contextual prompting supplies relevant background information that the model needs to answer the current task more accurately and appropriately.

Q: What is step-back prompting?

Step-back prompting first asks for broader principles or supporting context and then uses that information to solve the specific task.

Q: Does a good prompt eliminate hallucination?

No. Better prompts can reduce ambiguity and constrain outputs, but factual verification, grounding, retrieval, validation, and application-level controls may still be necessary.

21. Quick Revision Summary

- Zero-shot = no examples.
- One-shot = one example.
- Few-shot = multiple examples.
- System instructions define broad behavior and rules.
- Context provides relevant background.
- Role prompting guides perspective and style.
- Constraints define boundaries and reduce ambiguity.
- Step-back prompting first identifies broader principles or context.
- For complex tasks, request explicit, checkable stages and verification.
- Top-p and top-k are generation controls, not prompting methods.
- Structured output should use an explicit schema when machine readability matters.

- Prompt engineering improves task clarity but does not guarantee factual correctness.

Memory formula: Good Prompt = Clear Task + Relevant Context + Useful Constraints + Examples When Needed + Defined Output Format.